SMARTTASK: CONTEXT-AWARE MOBILE TO-DO LIST FOR INTELLIGENT TASK MANAGEMENT

Project ID: 25-26J-531

Project Proposal Report

De Silva S.J.W. – IT21228612

BSc (Hons) in Information Technology Specialising in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

SMARTTASK: CONTEXT-AWARE MOBILE TO-DO LIST FOR INTELLIGENT TASK MANAGEMENT

Project ID: 25-26J-531

Project Proposal Report

De Silva S.J.W. – IT21228612

BSc (Hons) in Information Technology Specialising in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

DECLARATION

I hereby declare that this proposal is entirely my own work and does not include, without proper acknowledgment, any material previously submitted for a degree or diploma at any other university or institution of higher learning. To the best of my knowledge and belief, it does not contain any material previously published or written by another person, except where appropriate acknowledgment is made in the text.

Student ID	Name	Date	Signature
IT21228612	De Silva S.J.W.	2025/11/05	

The candidate is conducting research for their undergraduate dissertation under our supervision.

	Title	Name	Date	Signature
Supervisor	Mr	Suresh Niroshan Fernando	2025/11/6	
Co-Supervisor	-	-	-	-
External Supervisor	-	-	-	-

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my Research Project Supervisor, Mr. Suresh Niroshan Fernando, for his continuous guidance, support, and encouragement during the preparation of this project proposal. His valuable advice has been essential in shaping the direction of this research.

I also wish to acknowledge all staff members of the Comprehensive Design and Analysis Project (CDAP) team for their constructive feedback and recommendations, which have been greatly helpful in refining this proposal.

Finally, I am thankful to my parents, friends, and colleagues for their constant encouragement and motivation throughout this stage of my academic journey.

ABSTRACT

Task management applications are widely used by students and professionals to organize daily responsibilities. However, most existing tools rely on static reminders and user-driven prioritization, which often leads to notification fatigue, missed deadlines, and increased cognitive load. The proposed solution, SmartTask, aims to address this problem by introducing a context-aware task management system that adapts intelligently to real-life conditions. The system will collect contextual signals such as location, time, device state, and user activity, and integrate them with task metadata to identify when a task is most relevant. Using custom prioritization algorithms and lightweight behavioral learning, SmartTask will dynamically rank tasks and deliver notifications at optimal moments, reducing interruptions while improving productivity.

The proposed system architecture includes modules for task input, context acquisition, context matching, task prioritization, adaptive notifications, and data-driven insights. A native Android implementation will be used to ensure efficient sensor access, offline capability, and seamless user interaction. The research design will include prototype development followed by evaluation using metrics such as task completion rate, reduction in missed reminders, user cognitive load (NASA-TLX), and perceived usability through user feedback surveys.

It is expected that SmartTask will demonstrate improved task adherence and reduced mental strain compared to traditional task managers. If successful, this approach can enhance productivity and well-being for users with demanding and dynamic schedules, showing strong potential for real-world adoption and future commercialization.

TABLE OF CONTENTS

Declaration i

Acknowledgement ii

Abstract iii

List of figures vi

List of tables vi

1 Introduction 1

2 Background & Literature Survey 4

3 Research Gap 11

3.1 Limitations of Commercial Tools 11

3.2 Limitations of Research Prototypes 11

3.3 Unaddressed Gaps in Current Literature 12

3.4 Advancement Goals and Creative Contribution of SmartTask 12

4 Research Problem 14

4.1 Specific Research Questions 14

4.2 Problem Statement 15

5 Objectives 17

5.1 Main Objective 17

5.2 Specific Objectives 17

6 Methodology 18

6.1 Introduction To Methodology 18

6.2 High-Level System Architecture 19

6.2.1 System Components and Interactions 21

6.2.2 Design Rationale and Expected Benefits 24

6.2.3 Self-Evaluation and Testing Plan 24

6.3 Technologies and Tools 26
6.4 Data Collection and Analysis 30
6.4.1 Required Data 30
6.4.2 Data Collection Methods 30
6.4.3 Data Preprocessing, Storage, and Analysis 30
6.4.4 Ethical Considerations 31
6.5 Tasks, Sub-Tasks, and Project Schedule 32
6.6 Work Breakdown Structure (WBS) 37
6.7 Gantt chart 38
7 Project Requirements 39
7.1 Functional requirements 39
7.2 Non-functional requirements 40
7.3 User requirements 41
7.4 System requirements 41
7.5 Use cases (Tentative) 42
7.5.1 Use Case 1: Install and Set Up the Mobile App 42
7.5.2 Use Case 2: Create and Manage Tasks 43
7.5.3 Use Case 3: Context Acquisition and Logging 43
7.5.4 Use Case 4: Dynamic Task Prioritization 44
7.5.5 Use Case 5: Adaptive Notification Delivery 45
7.5.6 Use Case 6: Behavioral Learning and Insights 45
7.6 Test Cases (Tentative) 47
8 Budget and Budget Justification 51
9 Ability of Commercialization 52
10 References 53

LIST OF FIGURES

Figure 6.1	High Level System Architecture Diagram	20
Figure 6.2	Components Flow In Real World	23
Figure 6.3	Work Breakdown Structure	37
Figure 6.4	Gantt Chart	38

LIST OF TABLES

Table 2.1	Comparison of ten popular commercial apps and ten research systems along key dimensions	7
Table 6.1	Technologies and tools used to develop the app	26
Table 6.2	Tasks , sub tasks and time durations for the project	32
Table 7.1	Test cases used to evaluate the system and their details	47
Table 8.1	Budget details of the project	51

1 INTRODUCTION

Modern knowledge workers face cognitive overload due to constant multitasking and digital interruptions. Empirical studies show that frequent interruptions and task-switching reduce performance and increase stress: for example, Mark et al. found that interrupted tasks took less total time but were completed with substantially higher stress, frustration and effort[1]. Gonzalez et al. similarly observed that users frequently switch between tasks and applications in rapid succession – often only a few minutes per task – creating "multi-tasking craziness" and cognitive strain[2]. These factors imply that presenting a flat, static to-do list (with all pending tasks shown equally) can overwhelm users' limited attention. For instance, the checklist-based approach of common task apps can become monotonous and disengaging, causing many users to abandon the app altogether[3]. Without intelligent filtering or prioritization, users must manually scan and decide which tasks matter most, adding to their mental burden.

At the same time, users commonly experience notification fatigue: habitual dismissals of reminders or alerts that feel irrelevant. While few academic studies explicitly quantify "fatigue," it is well known that extraneous notifications often go unnoticed or are ignored, reducing their effectiveness. For example, Winghart and Al Zaidy noted that conventional productivity apps "focus purely on task completion" without engagement mechanisms, and their static reminders can lead to user disengagement[3]. In practice, rigid pre-scheduled reminders often fire at suboptimal times, and users quickly tune them out.

Context-aware computing offers a way to alleviate these problems by adapting to the user's real-time situation. Context has been broadly defined as "any information that can be used to characterize the situation of an entity"[4]. In mobile computing, common contextual inputs include location, time, user activity (motion, calendar events), and device state (battery, connectivity). Surveys and frameworks have long emphasized that leveraging context can greatly improve mobile usability and effectiveness[5][6]. For instance, Baldauf et al. (2007) highlight that context-aware mechanisms (gathering sensor and user data) "in combination with mobile devices" can "increase usability tremendously"[5]. Modern context-aware platforms (e.g.

mobile sensing middleware) readily capture GPS location, activity recognition, and other signals. Indeed, recent systems like AwarNS demonstrate how a modular context framework can capture multi-sensor data and deliver timely interventions for health apps[6].

Despite this maturity in context sensing, most task-management tools on the market remain static and one-size-fits-all. Traditional to-do lists simply show all tasks and use fixed alerts, without adapting to context. This static approach neglects the rich contextual signals now available. For example, place-based reminder systems like Place-Its showed that tying tasks to location (e.g., “remind me at the grocery store”) can greatly improve usefulness[7], but still only use a single context dimension. Likewise, early prototypes such as Castaway (Kessell and Chan, CHI 2006) explored context-aware task views (e.g. mapping calendar events), yet did not dynamically combine multiple cues[8]. Other efforts (like Taskminder[9]) highlighted context in to-do lists, but were largely one-dimensional or research prototypes.

Existing approaches: Current research prototypes tend to focus on one context type or a static rule. Muñoz et al. (2010) built a mobile workflow app using priority, location and time to filter tasks[10], but their design did not fully automate adaptation or learn over time. Chihani et al. (2013) created DoTT, a rule-based mobile assistant allowing users to define IF–THEN context rules for app actions[11], but it relies on users writing rules rather than automated discovery. In summary, the state-of-the-art shows that context-awareness can enhance reminders and task UIs, but existing solutions usually handle only single-context triggers or manual rules, and are often limited to controlled (enterprise) settings rather than fluid daily life. In particular, they lack a holistic, multi-dimensional approach that learns a user’s habits and continuously prioritizes tasks.

Our approach and novelty: The proposed SmartTask system differs by integrating multiple contextual factors simultaneously (real-time location, time of day, user activity, device status, calendar context, etc.) to continuously reorder and trigger tasks in a mobile to-do list. By adapting which tasks are shown and when reminders fire based on the current context, SmartTask aims to reduce overload and fatigue. This builds directly on prior research (e.g., place-based reminders[7] and context

filtering[10]) but extends it by combining many signals and by using data-driven prioritization. For example, Zhang et al. (2022) recently demonstrated that personalized task prioritization (using past behavior and context history) can significantly reduce users' scheduling burden[12]. Similarly, our system will learn from a user's task completions and ignored alerts to refine future suggestions. In essence, SmartTask generalizes prior insights in a comprehensive, user-centric platform. We propose that by aligning task presentation and notifications with where, when, and how a user is working, we can improve adherence and productivity beyond what fixed to-do lists achieve. In sum, our introduction of a fully context-aware to-do list — which autonomously filters, ranks, and nudges tasks – represents a novel extension of existing work[10][12], with the potential to significantly mitigate cognitive load and notification fatigue.

2 BACKGROUND & LITERATURE SURVEY

Modern mobile *to-do list* apps overwhelmingly rely on static task lists and simple time-based reminders. For example, Todoist – "the world's #1 to-do list app" used by over 50 million people[13] – lets users capture tasks with natural-language input and set fixed due dates or location reminders, but offers no self-adjusting schedule or predictive reordering. It does provide **location-based notifications**, allowing arrival/departure triggers for tasks[14], and even *suggests* frequent locations to save time ("Todoist provides location suggestions based on your past location reminders"[14]). However, tasks themselves remain user-prioritized (static) and notifications fire exactly at their scheduled times or places. Any.do likewise provides multiple reminder types – task alerts, event alerts, and daily summary notifications – across platforms[15], but these are fixed reminders with no adaptation or learning. Microsoft To Do introduces a modest form of intelligence: its **"My Day"** view uses past behavior to *suggest* tasks each morning ("intelligent and personalized suggestions"[16]), but otherwise tasks and reminders are manually defined. By contrast, Google's offerings (Google Tasks and Google Keep) also lack dynamic adaptation: tasks simply notify at fixed times set by the user[17]. Google Keep until recently *had* location reminders, but even that is being removed[18].

Other popular tools focus on organization or collaboration rather than intelligence. TickTick offers rich reminder features – multiple countdown alerts, recurring schedules, and even **time and location reminders** ("set location-based reminders…get reminded when leaving or arriving"[19]) – yet it still treats task priorities as static user choices. Trello provides a visual Kanban interface for tasks[20], and Asana, Notion, Remember The Milk, etc., support list-based work or file attachments, but none inherently reorders tasks by urgency or user context. In short, most commercial apps are **context-unaware** (they do not sense the user's state or environment), use **static prioritization**, issue **fixed notifications**, and have little or no *behavioral learning*. These limitations are summarized in Table I below.

In contrast, academic research has increasingly explored **context-aware**, **adaptive** task management. Early prototypes like *Castaway* (Kessell and Chan, 2006) envisioned a mobile system to capture tasks on the fly and issue reminders "at precisely the right place and/or time"[8]. Castaway supported quick task entry, a lightweight map-based task view, and context-triggered notifications. Likewise, Spiteri's *ContextGear* (2009) used smartphone location to auto-generate reminders when users entered relevant areas. In collaborative settings, systems like *Overseer* leverage rich context for team tasks: Luqman and Griss demonstrate a mobile multi-agent system

that *"leverages context information in a mobile setting to facilitate collaboration and task allocation"* for disaster response, dynamically reassigning tasks to whoever is best positioned[21]. These research prototypes use **location, role, and activity context** to *adapt* task distribution, unlike rigid commercial tools.

A key stream of research focuses on **intelligent notifications**. Streefkerk *et al.* (2012) found that sensor-based context (location, activity, task phase) can predict "appropriate moments of interruption" and substantially reduce distraction[22]. Iqbal's OASIS framework (PhD 2008) goes further by detecting *task breakpoints* (cognitive transitions) and *deferring* notifications until those low-load moments. As described, "OASIS defers notifications until moments that have lower interruption costs," using models of task breakpoints[23]. Empirically, interrupting at breakpoints lowered disruption, giving *the first evidence of benefits of notification scheduling in authentic task settings*[23]. In other words, the timing of alerts is adaptively aligned with user context, whereas apps like Google Tasks notify without regard to the user's current activity[17].

Several systems employ on-device sensing and learning to drive adaptivity. Ahmed and Mujib (2014) built an Android "context-aware reminder" that uses the accelerometer and gyroscope to recognize activities (walking, jogging, etc.) and trigger task reminders accordingly[24]. The AWARE framework (2015) provides a comprehensive context-instrumentation toolkit: it continuously collects hardware, software, and social sensor data, and its *plugins* apply data mining and machine learning to "generate context" for applications[25]. While AWARE itself is a general platform, it illustrates how mobile contexts (location, movement, schedule, social input) can be abstracted for higher-level task intelligence. On a larger scale, Tian *et al.* (2022) analyzed real-world app-usage logs to uncover **multi-app task patterns**. They found common composite tasks (e.g. "dining out" spans multiple apps) and showed that next tasks could be predicted by learning from temporal and contextual features[26]. This kind of modeling could ultimately feed into to-do apps that anticipate the user's needs.

Other recent studies highlight both opportunities and pitfalls. The MageMemo app (Winghart and Al Zaidy, 2024) incorporated *gamification* into a mobile to-do list, boosting engagement but not exploiting context [3]. Kim *et al.* (IMWUT 2020) conducted an in-situ study of smartphone interruptions, identifying which contexts (e.g. location, activity) cause distraction and eliciting user "if-then" coping strategies[27]. Their findings suggest design rules (e.g. "if I arrive home, remind me to relax") for future task managers. In summary, the literature shows clear benefits

when task reminders and priority suggestions are **context-sensitive and behaviorally informed**, but no existing tool (commercial or prototype) fully realizes this.

Table 2.1 compares ten popular commercial apps and ten research systems along key dimensions. As the table shows, commercial apps largely lack context-awareness and learning (all are “Static” in prioritization and “Fixed” in notification timing). On the research side, various systems incorporate context (e.g. location-aware Castaway [8], collaboration-context Overseer [21]) and adaptive notifications (e.g. OASIS [23]), but often miss other elements (e.g. Castaway [8] has reminders at the right place/time but no automated re-prioritization).

Table 2.1 : comparison of ten popular commercial apps and ten research systems along key dimensions

Index	Name	Category	Context Awareness	Task Prioritization	Notification Adaptivity	Learning from Behavior	Notes
1	**Todoist**	Commercial	Partial (location)	Static	Fixed (time/location)	Yes (location suggestions)	Widely used; natural-language input[13], but only time/place triggers[14].
2	**Any.do**	Commercial	No	Static	Fixed	No	Offers standard reminders and daily summary alerts[15]; no context adaptivity.
3	**Microsoft To Do**	Commercial	No	Dynamic (My Day)	Fixed	Yes	Introduces “My Day” intelligent suggestions[16]; otherwise user-set priorities and alerts.
4	**Google Tasks**	Commercial	No	Static	Fixed	No	Bare-bones task list; only time/date reminders[17].
5	**TickTick**	Commercial	Yes (time/loc)	Static	Fixed	No	Multiple reminders (e.g. “1 day before”), supports time and *location* reminders[19].
6	**Trello**	Commercial	No	Static (board order)	Fixed	No	Kanban workflow; no smart scheduling, just due-date alerts[20].
7	**Remember The Milk**	Commercial	Yes (location)	Static	Fixed	No	Oldest list app; supports location reminders but no adaptive features.
8	**Notion**	Commercial	No	Static	Fixed	No	All-in-one workspace with checklists; lacks built-in adaptivity.
9	**Asana**	Commercial	No	Static	Fixed	No	Team task management; priorities set manually, notifications fixed.

10	**Google Keep**	Commercial	Partial (deprecated)	Static	Fixed	No	Note-taking app with reminders. Location reminders are being removed[18].
11	**Castaway**	Research	Yes (location/time)	Static	Adaptive (delay to context)	No	Prototype WIP[8]. Captures tasks quickly, reminds at right time/place.
12	**ContextGear**	Research	Yes (location)	Static	Fixed	No	Location-aware reminder prototype (Spiteri 2009).
13	**Overseer**	Research	Yes	Dynamic	Adaptive	Partial	Multi-agent system for disaster tasks[21]; uses context to reassign roles dynamically.
14	**Do Not Disturb (DND)**	Research	Yes (sensor-based)	N/A (interruptions)	Adaptive	Yes	Schedules mobile notifications at opportune breakpoints[22]; lowers interruption cost.
15	**MageMemo**	Research	No	Static	Fixed	Yes	Gamified to-do app (2024); improves engagement but is not context-aware.
16	**Context-Aware Reminder**	Research	Yes (activity)	Static	Adaptive	Yes	Android prototype uses accelerometer to detect activity for reminders[24].
17	**OASIS (Iqbal, 2008)**	Research	Yes (cognitive)	Static	Adaptive (breakpoints)	Yes	Desktop system detecting task breakpoints[23] to defer notifications; empirically reduces disruption.
18	**Task Prediction**	Research	Yes (usage context)	Dynamic	Adaptive	Yes	Tian *et al.* (2022): learns common multi-app tasks and predicts next tasks from context[26].

19	**AWARE**	Research	Yes	N/A (context platform)	N/A	Yes	Smartphone context-logging framework; plugins use data mining/ML to **generate context**[25].
20	**Kim *et al.* (2020)**	Research	Yes	N/A	Adaptive	Yes	Context-aware distraction study: derives if-then rules for proactive interruption management[27].

Despite this rich body of work, **no existing tool offers all the desirable features together**. Commercial apps miss context sensitivity and learning; research prototypes rarely combine all functionalities. SmartTask aims to fill these gaps by integrating multi-sensor context (location, activity, calendar) with a dynamic scheduling engine. In particular, SmartTask will learn from user behavior to reprioritize tasks (unlike the static schemes above) and adaptively time notifications to minimally interrupt the user (as per OASIS's insights[23]), while also offering rich reminders (e.g. based on location or historical patterns[13][26]). By bridging these dimensions, SmartTask addresses key limitations highlighted in Table 2.1 : it combines context-awareness, learning, and adaptive prompting in one mobile app. As such, SmartTask stands to provide more intelligent task management than any single existing commercial app or research system[23][19].

Justification: Existing task tools typically require the user to set priorities and alert times manually, leading to cognitive burden and missed opportunities. Context-aware research shows that smarter timing and personalized suggestions can significantly reduce interruptions and improve on-time task completion[23][26]. SmartTask will leverage these insights: it will sense when users are most receptive and automatically re-rank tasks based on learned habits. In effect, SmartTask unifies the adaptive notification scheduling of OASIS[23], the behavior-driven prediction of mobile tasks[26], and the everyday to-do management of apps like Todoist [14]. In doing so, SmartTask fills the critical gap of an intelligent, context-aware mobile to-do list that no current commercial app or research prototype fully provides.

3 RESEARCH GAP

Despite significant progress in mobile task management systems, both in commercial applications and academic prototypes, important gaps remain unaddressed. These gaps span four critical dimensions: context-awareness, task prioritization, notification adaptivity, and behavioral learning. Collectively, they highlight the absence of a **holistic, real-world solution** for mitigating cognitive overload and notification fatigue in daily task management.

3.1 Limitations of Commercial Tools

Commercial task managers such as Todoist, Any.do, Microsoft To Do, TickTick, and Google Tasks dominate productivity management, yet they rely heavily on **static prioritization and fixed reminders** [13]–[17]. Todoist, for example, allows natural language input and location-based triggers, but tasks remain user-prioritized, and notifications occur only at scheduled times or locations [13][14]. TickTick supports time and location reminders [19], while Microsoft To Do offers a limited "My Day" suggestion each morning [16]; nonetheless, dynamic adaptation and real-time reordering are absent.

This user-driven configuration imposes a **cognitive burden** on knowledge workers, who already operate under "constant multitasking craziness" [2]. Manually managing task priorities and reminders amplifies stress, reduces engagement, and undermines productivity [1][3]. Consequently, commercial apps fail to alleviate cognitive overload where support is most needed.

3.2 Limitations of Research Prototypes

Academic prototypes illustrate the potential of context-aware task management, but most remain **narrow, rule-based, or incomplete**. Castaway [8] supported location- and time-based triggers, yet its task views were static. ContextGear (Spiteri, 2009) extended location awareness but relied on single-context reminders, while Muñoz et al. [10] integrated priority, location, and time in a workflow app but lacked learning capabilities. DoTT [11] allowed IF–THEN rules but depended on user configuration.

Systems emphasizing intelligent notifications, such as OASIS [23] and Do Not Disturb [22], reduced interruption costs by aligning alerts with cognitive breakpoints, but these were **desktop- or lab-based** and not embedded in everyday mobile task management. Similarly, the Android activity-based reminder prototype [24] detected user motion but did not integrate broader prioritization, and task prediction models [26] remain at the modeling stage without practical deployment.

In essence, while research shows isolated benefits—location-based reminders [8], adaptive notifications [22][23], behavior prediction [26]—**no existing system unifies these into a mobile platform suitable for real-world use**.

3.3 Unaddressed Gaps in Current Literature

From the analysis above, four interrelated research gaps are evident:

1. **Multi-Context Integration** – Existing apps and prototypes often rely on a single context dimension (e.g., location [8][14], activity [24], or time [10]). Effective task management requires simultaneous awareness of user location, activity, time, calendar events, and device state [5], yet no tool operationalizes such multi-sensor integration.

2. **Dynamic Task Prioritization** – Static task ordering dominates both commercial and research solutions. While some apps allow manual reordering [13][15][20], none automatically reprioritize tasks based on real-time circumstances or past behavior. Evidence indicates that context-aligned prioritization significantly reduces scheduling burden [12], but this has yet to be implemented in practice.

3. **Adaptive Notification Timing** – Notification fatigue arises from rigid alerts that ignore user state [3][28]. OASIS [23] and Do Not Disturb [22] demonstrate adaptive timing in controlled settings, but mainstream apps continue to notify at fixed times regardless of activity [15][17], highlighting a gap between lab research and practical deployment.

4. **Learning from User Behavior** – Few systems continuously refine task suggestions based on user interactions. Microsoft To Do's "My Day" [16] and Todoist's location suggestions [14] provide only surface-level adaptation. More advanced behavior-driven models, such as Tian et al.'s task prediction [26] or Zhang et al.'s context-aware ranking [12], have not yet been embedded in mobile to-do lists.

3.4 Advancement Goals and Creative Contribution of SmartTask

SmartTask directly addresses the research gaps by offering :

- **Broad Context Integration:** SmartTask simultaneously captures multiple contextual signals, including location, time, user activity, calendar events, and device state, going beyond single-dimension triggers [5][8][24].

- **Dynamic Task Prioritization:** Tasks are automatically reordered based on real-time context and past user behavior, providing a personalized and adaptive task sequence that reduces cognitive load [12].
- **Adaptive Notification Timing:** Reminders and nudges are delivered at moments when users are most receptive, minimizing interruption and notification fatigue, inspired by insights from OASIS [23] and in-situ interruption studies [27][22].
- **Behavioral Learning:** The system continuously learns from user interactions, refining future task ordering and notification timing to better align with individual habits and preferences [26].

By combining multi-context awareness, context-sensitive prioritization, intelligent notifications, and continuous learning, SmartTask **bridges the gap between static commercial apps [13][17][19] and research prototypes that address only isolated features [8][24][23]**. This integrated design aims to **reduce cognitive overload, enhance task adherence, and improve productivity** in real-world, multitasking environments [1][2][23][26].

4 RESEARCH PROBLEM

4.1 Specific Research Questions

1. How can mobile task management systems effectively integrate multiple contextual signals to enhance task prioritization?

Current commercial apps and research prototypes often rely on a single context dimension, such as location [8][14] or activity [24], and use static task ordering [13][15][20]. This approach does not account for the complexity of real-world multitasking environments, where users' location, activity, time availability, calendar events, and device status collectively influence task relevance [5]. This research aims to investigate methods for dynamically combining these signals to automatically reorder tasks, reducing cognitive overload and supporting timely task completion.

2. How can notifications be adaptively scheduled to minimize distraction and notification fatigue?

Pre-scheduled alerts in most apps fail to consider users' cognitive load, leading to ignored or disruptive notifications [3][28][15][17]. Prior research, such as OASIS [23] and Do Not Disturb [22], shows that aligning alerts with cognitive breakpoints or contextual cues improves effectiveness. This problem explores how adaptive notification strategies can be implemented in a mobile task management system, triggering reminders when users are most receptive while avoiding unnecessary interruptions.

3. How can user behavior be leveraged to personalize task management and improve adherence?

Existing apps provide limited behavioral learning, typically relying on simple heuristics like frequent locations or daily suggestions [13][16]. Advanced models for task prediction and context-aware ranking [12][26] have yet to be embedded in everyday mobile to-do lists. This research examines how continuous learning from

task completion patterns, ignored alerts, and contextual interactions can refine task prioritization and notification timing for each user.

4. How can a holistic, context-aware mobile task management system be designed for practical use?

Research prototypes demonstrate isolated benefits—location-based reminders [8], activity-triggered alerts [24], or predictive sequencing [26]—but no system combines these features into a unified, mobile-ready application. This question seeks to develop design principles, algorithms, and integration strategies that deliver a fully context-aware, adaptive, and behaviorally intelligent task management system capable of addressing the limitations of both commercial apps and existing prototypes [1][2][23][26].

4.2 Problem Statement

Modern knowledge workers face significant cognitive overload due to constant multitasking, digital interruptions, and static task management systems [1][2]. While commercial task management apps such as Todoist, TickTick, and Microsoft To Do offer reminders and simple prioritization [13][16][19], they fail to dynamically adapt to users' real-world contexts. Existing research prototypes demonstrate the potential of context-aware task management—using location, activity, or temporal cues to trigger reminders [8][24][23]—but these systems are often limited to single-context triggers, isolated adaptive features, or controlled laboratory settings [22][26].

Consequently, users are left with systems that require manual prioritization, fire notifications at suboptimal moments, and provide minimal behavioral learning. This gap results in inefficient task completion, overlooked reminders, and heightened cognitive burden [3][28]. The problem lies in the lack of a holistic mobile solution that can simultaneously integrate multiple contextual signals, dynamically reorder tasks,

adapt notification timing to cognitive load, and continuously learn from user behavior [5][12][26].

The research challenge, therefore, is to design and implement a fully context-aware, adaptive, and behaviorally intelligent mobile task management system that addresses these limitations. Specifically, the system must: (i) integrate multi-dimensional contextual signals such as location, activity, calendar events, and device status; (ii) dynamically prioritize tasks in real-time based on user context; (iii) adapt notification schedules to minimize distraction and fatigue; and (iv) continuously learn from user interactions to improve task recommendations. By tackling these issues, SmartTask aims to reduce cognitive overload, mitigate notification fatigue, and enhance productivity in everyday mobile task management scenarios [1][2][8][23][26].

5 OBJECTIVES

5.1 Main Objective

To design and implement a context-aware mobile task management system, SmartTask, that dynamically prioritizes tasks, adapts notifications based on user context, and continuously learns from user behavior to reduce cognitive overload and notification fatigue in everyday mobile task management**.**

5.2 Specific Objectives

1. **Contextual Signal Integration:** Identify and integrate multiple contextual inputs—such as location, time, user activity, calendar events, and device status—into the task management system to support intelligent decision-making.
2. **Dynamic Task Prioritization:** Develop algorithms that automatically reorder tasks in real-time based on the user's contextual signals and historical task completion patterns.
3. **Adaptive Notification Scheduling:** Implement a notification system that triggers alerts at moments of low cognitive load, minimizing distractions and notification fatigue.
4. **Behavioral Learning:** Continuously monitor user interactions (e.g., completed tasks, ignored reminders) and refine task prioritization and notification timing accordingly.
5. **Evaluation of System Effectiveness:** Assess the effectiveness of SmartTask in real-world scenarios by measuring task completion efficiency, user engagement, reduction in missed reminders, and perceived cognitive load.
6. **Broad System Design:** Design a unified, mobile-ready platform that combines all above functionalities into a practical, user-centric application suitable for everyday task management.

6 METHODOLOGY

6.1 Introduction To Methodology

The methodology of this study focuses on the design and development of a context-aware mobile task management system, SmartTask, with the primary goal of reducing cognitive overload and notification fatigue. This approach integrates principles from context-aware computing, mobile application development, and lightweight machine learning, ensuring that the solution is both technically feasible and user-centric.

This methodology is particularly appropriate because mobile devices are uniquely capable of capturing real-time contextual signals such as location, activity, time availability, calendar events, and device status, which are essential for adaptive task management. By leveraging these signals, SmartTask dynamically prioritizes tasks, schedules notifications at contextually optimal moments, and continuously learns from user behavior to personalize recommendations.

The chosen research area combines context-aware computing, which enables systems to adapt based on user environment and behavior, with behavioral modeling and lightweight machine learning, which supports dynamic task ranking and personalized notifications. These pillars are critical for addressing the core problem of task overload in multitasking environments, as they allow the system to sense, predict, and respond intelligently to user needs.

The development process follows a modular design approach, where each component is implemented independently before integration into a unified mobile application. This ensures clarity of system functionality, ease of testing, and flexibility for future enhancements.

Furthermore, the methodology emphasizes practical evaluation, combining system performance metrics (e.g., task completion rate, missed reminders) with user-centered measures (e.g., perceived cognitive load, satisfaction). This dual approach allows both technical efficiency and real-world usability to be assessed.

Overall, this methodology provides a structured path to achieving the research objectives, aligning system design, technological choices, and evaluation methods with the core aim of enhancing productivity in real-world multitasking environments. By situating the study within context-aware computing and leveraging behavioral modeling principles, the approach ensures that SmartTask is both innovative and grounded in established research frameworks.

6.2 High-Level System Architecture

The figure 6.1 attached below displays the **High-Level System Architecture** of SmartTask. It provides a conceptual overview of how the system's modules interact to achieve context-aware task management and intelligent notifications. The architecture is designed to ensure modularity, scalability, and continuous learning, aligning with the core objectives of reducing cognitive overload and notification fatigue.

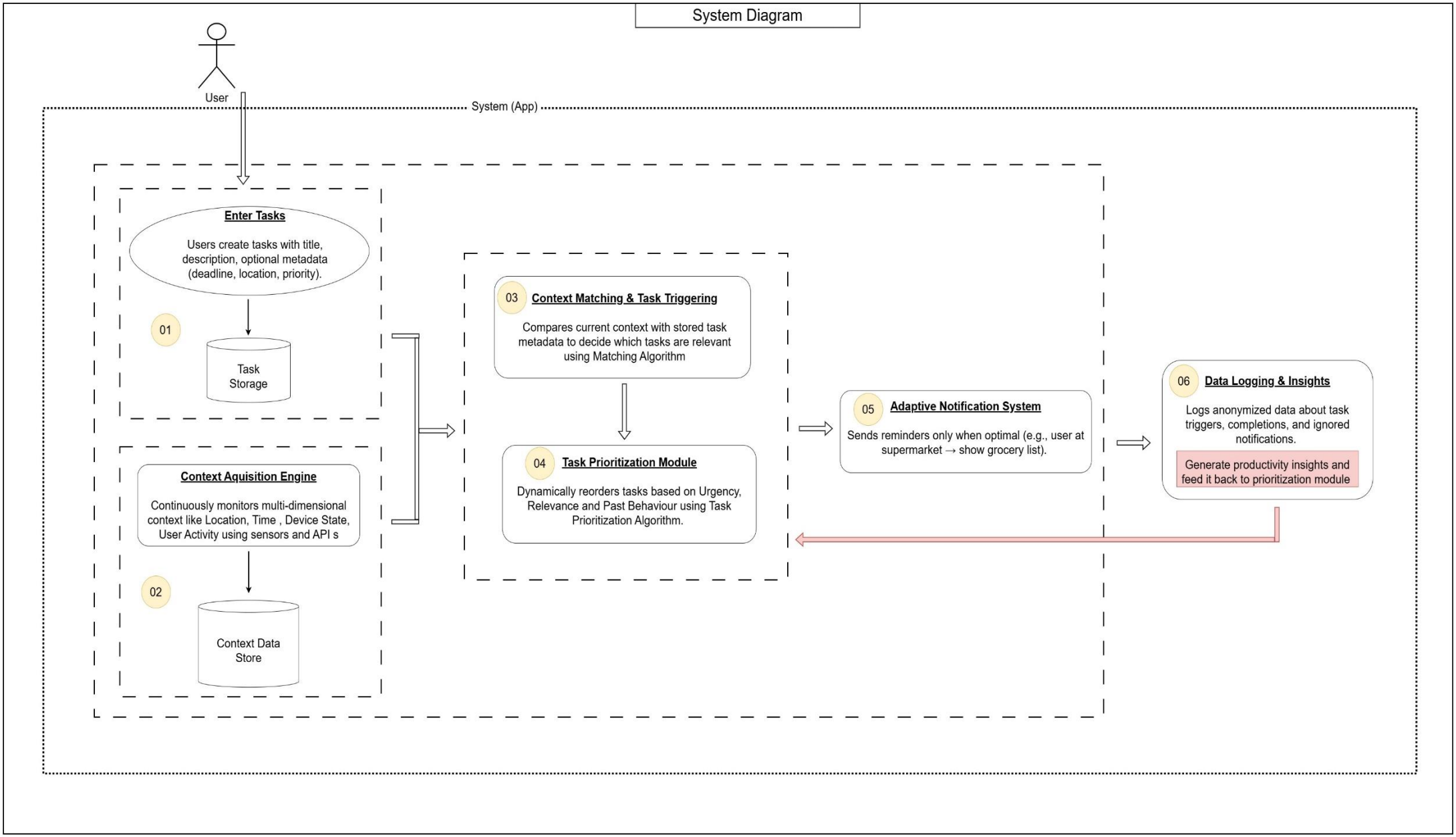

Figure 6.1 : High-Level System Architecture of SmartTask

6.2.1 System Components and Interactions

1. **Task Input Module**
    - Users create tasks with relevant metadata: title, description, deadline, location, and priority.
    - Metadata is stored in a **Task Database** for subsequent use in context matching.
2. **Context Acquisition Engine**
    - Continuously collects real-time contextual signals:
        - **Location:** via Google Maps API geofencing.
        - **Time:** system clock and calendar events.
        - **Device State:** battery level, connectivity, and other system metrics.
        - **User Activity Patterns:** motion sensors and scheduled events.
    - Stores all contextual data in the **Context Data Store**.
3. **Context Matching & Task Triggering**
    - Matches current context with stored task metadata to identify relevant tasks.
    - Generates a **filtered task list** for display based on real-time conditions.
4. **Task Prioritization Module**
    - Dynamically orders tasks by urgency, contextual relevance, and historical behavior.
    - Works in feedback loop with **Data Logging & Insights** to continuously improve prioritization.

5. **Adaptive Notification System**
    - Pushes context-aware reminders and nudges based on the prioritized task list.
    - Minimizes notification fatigue by delivering alerts at optimal moments.
6. **Data Logging & Insights**
    - Captures user responses (completion, dismissal, postponement) and contextual conditions.
    - Generates analytics to refine prioritization and notification timing.
    - Feeds insights back to the **Task Prioritization Module** for continuous learning.

This architecture ensures that SmartTask can intelligently sense context, prioritize tasks, deliver adaptive notifications, and continuously improve over time, resulting in enhanced user productivity and reduced cognitive strain. Figure 6.2 attached below displays the Components Flow in Real World.

Scenario: A student has 3 upcoming tasks today:

1. Finish Assignment – Deadline: 2:00 PM | Location: Library | Priority: High
2. Buy Stationery – Deadline: Evening | Location: Bookshop | Priority: Medium
3. Call Family – Deadline: Night | Location: Home | Priority: Low

1 - Task Input Module

- Student enters tasks with metadata: title, deadline, location, priority.
- Example: "Finish Assignment – 2:00 PM – Library – High Priority".
- Stored in Task Database.

2 - Context Acquisition Engine

- Continuously gathers real-time signals:
  - Location → detects when student is at home or library.
  - Time → checks system clock (e.g., 1:30 PM, 30 mins before deadline).
  - Device state → battery at 20% triggers early reminder.
  - Activity → detects "walking" → student is moving toward library.
- Stored in Context Data Store.

3 - Context Matching & Task Triggering

- Matches current context with task metadata.
- Example: At 1:30 PM, location = near library, time = 30 mins to deadline.
- Filters task list → highlights "Finish Assignment" as relevant.

4 - Task Prioritization Module

- Orders tasks dynamically:
  - Finish Assignment → top priority (deadline + location match).
  - Buy Stationery → lower (not urgent now).
  - Call Family → lowest (later at night).
- Works in loop with insights to refine ranking.

5 - Adaptive Notification System

- Sends just-in-time reminder:
  - "You're near the library, time to finish your assignment before 2:00 PM."
- Suppresses irrelevant notifications (Call Family won't pop up now).

6 - Data Logging & Insights

- Logs: student opened the reminder, completed assignment at 1:55 PM.
- Stores response + context.
- Feeds back into prioritization → system learns optimal reminder timing (e.g., 30 mins before worked well).

Result:

- Student finishes assignment on time with minimal distractions.
- Notifications felt timely and useful.
- System adapts to user behavior for future days.

Figure 6.2 : Components Flow in Real World

6.2.2 Design Rationale and Expected Benefits

- **Modularity:** Each component functions independently, allowing easier development, testing, and future enhancements.
- **Real-Time Adaptation:** Continuous context sensing ensures tasks and reminders align with the user's current state.
- **Behavioral Learning:** Feedback from the Data Logging module enables the system to personalize task presentation and notification schedules over time.
- **Cognitive Load Reduction:** By filtering and prioritizing tasks dynamically, the system minimizes user effort in scanning and selecting tasks.
- **Notification Fatigue Mitigation:** Adaptive alerts reduce interruptions, ensuring reminders are relevant and actionable.

6.2.3 Self-Evaluation and Testing Plan

To make sure SmartTask works well and improves productivity, a few key tests and evaluations will be done.

- **Task Completion Rate:** This checks how many tasks users finish on time compared to how many are missed or postponed. A higher completion rate means the app is helping users stay organized and meet their deadlines.
- **Reduction in Missed Reminders:** SmartTask's adaptive reminders will be compared with normal fixed-time reminders. The idea is to see if SmartTask's context-based alerts (like time, location, or activity) help reduce the number of missed reminders.
- **User Cognitive Load (NASA-TLX):** The NASA Task Load Index survey will be used to measure how mentally demanding the app feels to use. It asks users about things like effort, frustration, and how much mental workload they experienced while using SmartTask.

- **User Feedback / Satisfaction:** A short survey will be shared with users to get their opinions on how easy, useful, and effective the app feels. This helps identify if the app actually makes people feel more productive and less stressed.
- **Simulation and Real-World Testing:** The app will first be tested using the Android Emulator to simulate different conditions (like location changes, calendar events, and battery levels). After that, it will be tested on real Android devices to see how it performs in daily situations — for example, sending reminders when a user reaches a specific place or starts a certain activity.

6.3 Technologies and Tools

Table 6.1 below compares and justifies the tools and technologies used to develop the app.

Table 6.1 : Tools and technologies used to develop the app.

#	Task / Requirement	Available Technologies / Tools	Selected Technology & Justification
1	**Mobile App Development**	Java, Kotlin, Flutter, React Native, Swift	**Java (Android native app)** – Easy to learn, widely supported by community, allows full access to Android sensors and APIs, ensures high performance. **Downside of others:** Kotlin slightly steeper learning curve; Flutter/React Native limit direct sensor access; Swift is iOS-only.
2	**Context Acquisition (Sensors & APIs)**	Google Maps API, Android Location Services, Activity Recognition API, Calendar API, Bluetooth/Wi-Fi APIs	**Google Maps API + Android native APIs** – Reliable, accurate geofencing, motion/activity recognition, calendar & device state capture. **Downside of others:** Cross-platform libraries may limit sensor access or introduce latency.
3	**Context Data Store & Task Metadata**	SQLite, Room, Firebase Realtime Database, MongoDB Atlas	**SQLite / Room** – Stores task attributes and contextual logs; lightweight, fast, fully integrated. **Downside of others:** Firebase requires network; MongoDB Atlas is overkill for mobile-local storage.

4	**Context Matching & Task Triggering**	Java libraries, Lightweight ML libraries (Weka, DL4J), Rule engines	**Java custom logic / lightweight ML** – Efficient for matching tasks to context; fully compatible with Android native environment. **Downside of others:** Full ML frameworks may be heavy for mobile-only processing.
5	**Task Prioritization Module**	Java-based ML libraries, custom algorithms, rule engines	**Custom scoring algorithms in Java** – Simple, lightweight, allows dynamic ranking based on urgency, relevance, and user behavior. **Downside of others:** Full ML frameworks can be overkill for mobile app; hybrid solutions may complicate integration.
6	**Adaptive Notification System**	Android Notification API, Firebase Cloud Messaging	**Android Notification API** – Provides native, context-aware notifications and nudges; works offline and integrates with device state. **Downside of others:** FCM requires network; may not be ideal for local adaptive notifications.
7	**Data Logging & Insights**	SQLite/Room, JSON, Firebase Analytics, Google Analytics, Java visualization libraries	**SQLite / Room + optional Java libraries** – Logs task triggers, completions, and ignored notifications; enables analysis and learning. **Downside of others:** Firebase/Google Analytics can compromise privacy and add complexity.
8	**Insight Generation & Behavior Pattern Analysis**	Java Collections & Streams, SQLite queries, Apache Commons Math, Weka, Deeplearning4j (DL4J)	**Java custom logic + SQLite queries –** Efficient for analyzing task completion, ignoring patterns, and notification timing on-device; lightweight and fully compatible.

			Optional: Weka or DL4J if needed to implement more advanced ML for pattern recognition. **Downside of others:** Full ML frameworks like DL4J are heavier, may increase app size and processing load.
9	**Testing / Simulation**	Android Emulator, Genymotion, Physical Android Devices	**Android Emulator + Physical Devices** – Emulator for multi-version testing; real devices for validating sensors, notifications, and performance. **Downside of others:** Genymotion may not fully replicate all sensor data and some features are paid.
10	**Testing Libraries / Frameworks**	JUnit, Espresso, Robolectric, Mockito	**JUnit + Espresso + Mockito** – JUnit for unit testing logic, Espresso for UI/interactions, Mockito for mocking dependencies; fully compatible with Android Studio. **Downside of others:** Robolectric is limited for full integration testing with sensors.
11	**IDE / Development Environment**	Android Studio, Eclipse, VS Code	**Android Studio** – Official Android IDE, full support for Java, debugging, emulator, Gradle build, and sensor integration. **Downside of others:** Eclipse outdated; VS Code needs extra setup for native Android.
12	**Version Control / CI**	Git, GitHub, GitLab	**Git + GitHub** – Widely used, integrates with CI/CD, enables collaboration and version tracking.

			Downside of others: GitLab less integrated with Android workflows.
13	**Graphics / UI Mockups**	Figma, Adobe XD, Sketch	**Figma** – Web-based, collaborative, free-tier available, easy UI/UX design before implementation. **Downside of others:** Adobe XD/Sketch require licenses, platform restrictions.
14	**Hardware Requirements**	Android smartphones (various versions), Windows/Mac computer	**Android smartphones + PC with Android Studio** – Real devices needed to test sensors, notifications; computer required for development and emulator.

6.4 Data Collection and Analysis

6.4.1 Required Data

The SmartTask system requires several types of data to function effectively:

- **User Activity Patterns** – Captured from device sensors (accelerometer, gyroscope) and calendar events to understand motion and availability.
- **Task Completion Logs** – Records of task creation, completion time, postponements, and ignored tasks.
- **Notification Interactions** – Logs of user responses to reminders and nudges, including dismissals or engagement.
- **Contextual Data** – Location, time, device state (battery level, connectivity), and other environmental signals at the time of task triggers.

6.4.2 Data Collection Methods

- **Sensors & APIs:** Android device sensors (motion, location, battery) and system APIs (calendar, device state) automatically collect real-time context data.
- **Application Logs:** Task metadata, completion events, and notification responses are stored locally in SQLite/Room database.
- **Optional Surveys/Interviews:** To supplement automated logs, user surveys or interviews may gather subjective feedback on cognitive load, satisfaction, and perceived relevance of notifications.

6.4.3 Data Preprocessing, Storage, and Analysis

- **Preprocessing:** Raw sensor readings are cleaned and filtered to remove noise (e.g., minor location fluctuations, false motion triggers). Timestamp alignment ensures context and task events are accurately synchronized.
- **Storage:** Processed data is stored securely in a local SQLite/Room database, ensuring fast access and persistence. Sensitive data is anonymized to protect user privacy.

- **Analysis:** Custom Java algorithms process task completion patterns, notification engagement, and contextual relevance. Aggregated insights inform dynamic task prioritization and adaptive notification scheduling. Optional lightweight ML libraries (e.g., Weka) may be used for predictive behavior modeling.

6.4.4 Ethical Considerations

SmartTask has been designed with strong ethical principles to ensure user privacy, transparency, and responsible data usage.

- **Local Storage Only:** All task and context data are stored securely on the user's device using local databases (SQLite/Room), ensuring no external transmission of personal information.
- **Anonymized Data:** Any data used for analytics or behavioral learning is anonymized before storage, preventing identification of individual users.
- **User Control:** Users retain full control over their data and can delete all stored tasks, context records, and app data at any time through the app's settings.
- **User Consent:** Explicit user consent is obtained during the initial app setup via a privacy policy agreement that clearly explains what contextual and activity data are collected and why.
- **Minimal Battery Impact:** Context sampling and background activity monitoring are optimized to minimize power consumption, ensuring ethical use of device resources.
- **Purpose Limitation:** Collected data is used solely to enhance task prioritization accuracy and notification relevance. No data is shared externally or used for third-party purposes.

6.5 Tasks, Sub-Tasks, and Project Schedule

The development of SmartTask is divided into major tasks with detailed sub-tasks, durations, and milestones. Each sub-task has a realistic estimated duration to ensure completion within the 6-month timeline. A Gantt chart or WBS can visually represent this schedule. Table 6.2 lists the tasks , sub tasks and time durations for the project.

Table 6.2 : Tasks , sub tasks and time durations for the project

#	Major Task	Sub-Tasks	Estimated Duration (per sub-task)	Total Duration	Milestones / Expected Completion
1	Requirements Refining & Analysis	Refine gathered requirements + Gather requirements further	5 days	2 weeks (Oct 1– Oct 14, 2025)	Requirements specification completed – Oct 14, 2025
		Further analyze requirements	5 days		
		Document user scenarios	4 days		

2	Background Study & Environment Setup	Study context-aware computing, mobile ML, adaptive notifications	6 days	2 weeks (Oct 15–Oct 28, 2025)	Knowledge foundation established, environment ready – Oct 28, 2025
		Install IDEs, libraries, SDKs, and tools	4 days		
		Set up Android Studio, emulator, and devices	4 days		
3	Wireframes & UI/UX Sketches	Prepare task input, dashboard, notifications, and settings screens	6 days	2 weeks (Oct 29–Nov 11, 2025)	Wireframes finalized – Nov 11, 2025
		Review and refine wireframes	4 days		
		Finalize UI flow	4 days		
4	Context Acquisition	Integrate location, motion, battery, and calendar sensors	2 weeks	4 weeks (Nov 12–Dec 9, 2025)	Sensor & API integration completed – Dec 9, 2025
		Implement Google Maps API and Activity Recognition API	1 week		
		Capture and preprocess contextual data	1 week		
		Store context in local database	1 week		

5	Task Input & Metadata Management	Develop task creation interface	1 week	3 weeks (Dec 10–Dec 30, 2025)	Task input module ready – Dec 30, 2025
		Validate input and store metadata	1 week		
		Integrate with context data	1 week		
6	Context Matching & Task Triggering	Retrieve tasks based on context	5 days	2 weeks (Dec 31, 2025–Jan 13, 2026)	Context-based task filtering implemented – Jan 13, 2026
		Filter and rank tasks according to relevance	5 days		
		Generate dynamically updated task list	4 days		
7	Task Prioritization Module	Develop urgency and relevance-based ranking algorithm	1 week	2 weeks (Jan 14–Jan 27, 2026)	Prioritization algorithm ready – Jan 27, 2026
		Integrate past user behaviour logs	3 days		
		Enable dynamic updates to task order	4 days		
8	Adaptive Notification System	Implement context-aware notification logic	5 days	2 weeks (Jan 28–Feb 10, 2026)	Adaptive notifications functional – Feb 10, 2026
		Schedule notifications at optimal times	4 days		

		Track user responses	5 days		
9	Data Logging & Insight Generation	Log task completion, dismissals, and context	4 days	2 weeks (Feb 11–Feb 24, 2026)	Data analytics integrated – Feb 24, 2026
		Analyze behavior patterns	5 days		
		Feed insights back into prioritization module	5 days		
10	Testing & Validation	Unit testing (JUnit)	5 days	3 weeks (Feb 25–Mar 16, 2026)	Testing and validation completed – Mar 16, 2026
		UI and interaction testing (Espresso)	4 days		
		Mocking and integration testing (Mockito)	5 days		
		User evaluation for cognitive load and satisfaction	5 days		
11	Deployment & Documentation	Final integration of all modules	6 days	2 weeks (Mar 17–Mar 31, 2026)	
		Deploy to Android devices	2 days		

		Prepare project report , Research Paper, and user manual	6 days		Final deployment and documentation – Mar 31, 2026

Notes on Timeline and Evaluation:

- Tasks are adjusted and overlapped where possible to fit the 6-month window.
- Iterative development is maintained for tasks 4–9 to allow testing after each sub-task.
- Milestones are defined at the end of each sub-task for early feedback and progress tracking.
- User evaluation metrics, including task completion rate, notification effectiveness, cognitive load, and satisfaction, are incorporated into testing.

6.6 Work Breakdown Structure (WBS)

Figure 6.3 attached below illustrates the Work Breakdown Structure of the entire project.

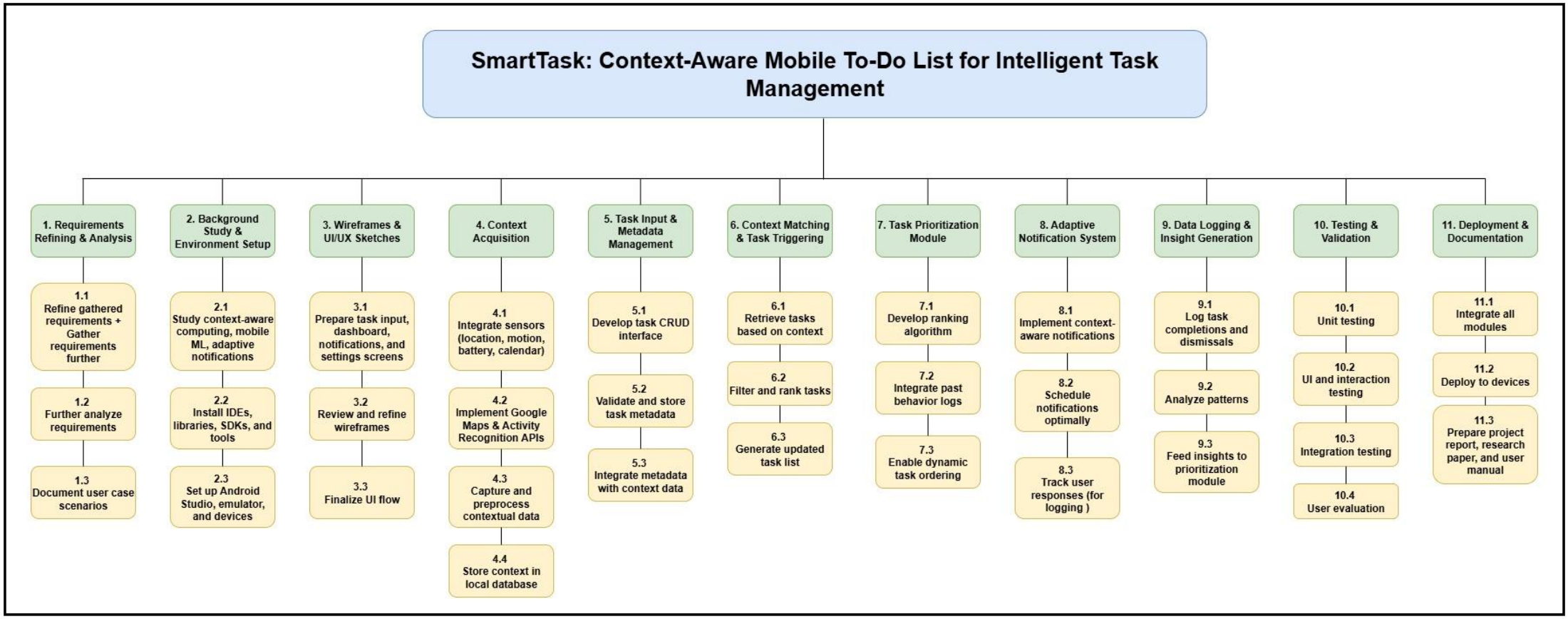

Figure 6.3 : Work Breakdown Structure (WBS) of SmartTask project

6.7 Gantt chart

Figure 6.4 attached below illustrates the Gantt Chart of the entire project.

Figure 6.4 : Gantt Chart of the entire project.

#	Task / Sub-Task	Start	End	Duration
1	**Requirements Refining & Analysis (2 weeks)**			
	1.1 Refine gathered requirements + Gather requirements further	Oct 1, 2025	Oct 7, 2025	5 days
	1.2 Further analyze requirements	Oct 8, 2025	Oct 14, 2025	5 days
	1.3 Document user scenarios	Oct 10, 2025	Oct 13, 2025	4 days
2	**Background Study & Environment Setup (2 weeks)**			
	2.1 Study context-aware computing, mobile ML, adaptive notifications	Oct 15, 2025	Oct 22, 2025	6 days
	2.2 Install IDEs, libraries, SDKs, and tools	Oct 23, 2025	Oct 28, 2025	4 days
	2.3 Set up Android Studio, emulator, and devices	Oct 25, 2025	Oct 28, 2025	4 days
3	**Wireframes & UI/UX Sketches (2 weeks)**			
	3.1 Prepare task input, dashboard, notifications, and settings screens	Oct 29, 2025	Nov 3, 2025	6 days
	3.2 Review and refine wireframes	Nov 4, 2025	Nov 7, 2025	4 days
	3.3 Finalize UI flow	Nov 8, 2025	Nov 11, 2025	4 days
4	**Context Acquisition (4 weeks)**			
	4.1 Integrate location, motion, battery, and calendar sensors	Nov 12, 2025	Nov 25, 2025	2 weeks
	4.2 Implement Google Maps API and Activity Recognition API	Nov 24, 2025	Dec 2, 2025	1 week
	4.3 Capture and preprocess contextual data	Dec 3, 2025	Dec 9, 2025	1 week
	4.4 Store context in local database	Dec 3, 2025	Dec 9, 2025	1 week
5	**Task Input & Metadata Management (3 weeks)**			
	5.1 Develop task creation interface	Dec 10, 2025	Dec 16, 2025	1 week
	5.2 Validate input and store metadata	Dec 17, 2025	Dec 23, 2025	1 week
	5.3 Integrate with context data	Dec 24, 2025	Dec 30, 2025	1 week
6	**Context Matching & Task Triggering (2 weeks)**			
	6.1 Retrieve tasks based on context	Dec 31, 2025	Jan 6, 2026	5 days
	6.2 Filter and rank tasks according to relevance	Jan 7, 2026	Jan 13, 2026	5 days
	6.3 Generate dynamically updated task list	Jan 8, 2026	Jan 13, 2026	4 days
7	**Task Prioritization Module (2 weeks)**			
	7.1 Develop urgency and relevance-based ranking algorithm	Jan 14, 2026	Jan 20, 2026	1 week
	7.2 Integrate past user behavior logs	Jan 21, 2026	Jan 23, 2026	3 days
	7.3 Enable dynamic updates to task order	Jan 24, 2026	Jan 27, 2026	4 days
8	**Adaptive Notification System (2 weeks)**			
	8.1 Implement context-aware notification logic	Jan 28, 2026	Feb 3, 2026	5 days
	8.2 Schedule notifications at optimal times	Feb 4, 2026	Feb 7, 2026	4 days
	8.3 Track user responses	Feb 5, 2026	Feb 10, 2026	5 days
9	**Data Logging & Insight Generation (2 weeks)**			
	9.1 Log task completion, dismissals, and context	Feb 11, 2026	Feb 14, 2026	4 days
	9.2 Analyze behavior patterns	Feb 15, 2026	Feb 19, 2026	5 days
	9.3 Feed insights back into prioritization module	Feb 20, 2026	Feb 24, 2026	5 days
10	**Testing & Validation (3 weeks)**			
	10.1 Unit testing (JUnit)	Feb 25, 2026	Mar 2, 2026	5 days
	10.2 UI and interaction testing (Espresso)	Mar 3, 2026	Mar 6, 2026	4 days
	10.3 Mocking and integration testing (Mockito)	Mar 7, 2026	Mar 11, 2026	5 days
	10.4 User evaluation for cognitive load and satisfaction	Mar 12, 2026	Mar 16, 2026	5 days
11	**Deployment & Documentation (2 weeks)**			
	11.1 Final integration of all modules	Mar 17, 2026	Mar 22, 2026	6 days
	11.2 Deploy to Android devices	Mar 23, 2026	Mar 24, 2026	2 days
	11.3 Prepare project report, Research Paper, and user manual	Mar 25, 2026	Mar 31, 2026	6 days

7 PROJECT REQUIREMENTS

7.1 Functional requirements

Functional requirements describe the **specific behaviors and functions** that the system must provide to meet user needs and research objectives.

1. **Task Management**
    - Users can create, edit, and delete tasks with metadata including title, description, deadline, location, and priority.
    - Tasks can be grouped or categorized (e.g., work, personal, urgent) to support context-aware filtering.
    - Task metadata must support dynamic context matching (location, time, device state, user activity).
2. **Context Acquisition**
    - Continuous collection of multi-context data: user location (via GPS), activity (walking, running, idle), time, calendar events, and device state (battery, connectivity).
    - Preprocessing of raw data to remove noise (e.g., minor location fluctuations) and synchronize timestamps for accurate task triggering.
3. **Context Matching & Task Triggering**
    - Identify relevant tasks based on current context.
    - Generate and update a filtered task list in real-time, considering multiple contextual signals simultaneously.
4. **Dynamic Task Prioritization**
    - Automatically reorder tasks based on urgency, contextual relevance, and historical completion patterns.
    - Support manual reordering while retaining context-driven recommendations.
    - Adapt prioritization dynamically as context changes (e.g., user moves to a work location or a meeting begins).
5. **Adaptive Notification System**
    - Deliver notifications and nudges based on task priority, user activity, and cognitive load.
    - Suppress notifications during inappropriate contexts (e.g., during meetings or device idle periods).
    - Record user interaction with notifications (dismissed, completed, or postponed) for learning purposes.
6. **Behavioral Learning & Analytics**
    - Continuously update task ranking and notification timing based on user interactions.
    - Generate insights such as task completion trends, ignored reminders, and context-sensitive behavioral patterns.
    - Provide optional analytics dashboard for users to review productivity and engagement metrics.

7. **User Interface & Experience**
    - Provide intuitive interfaces for task input, task lists, notification settings, and activity summaries.
    - Support smooth navigation, responsive design, and clear feedback for user actions.

7.2 Non-functional requirements

Non-functional requirements define **how the system should perform** and additional constraints, focusing on reliability, usability, and maintainability.

1. **Performance**
    - Context acquisition and task prioritization must occur with minimal delay (< 1 second latency) to ensure real-time relevance.
    - Notifications should trigger promptly based on contextual changes.
2. **Usability**
    - Intuitive, minimalistic interface suitable for daily use by knowledge workers.
    - Clear feedback mechanisms for task creation, completion, and notifications.
3. **Scalability**
    - Capable of handling large numbers of tasks and frequent context updates without performance degradation.
    - Modular architecture to allow future extension (e.g., integration with cloud storage or cross-platform support).
4. **Reliability & Availability**
    - System must handle sensor or API failures gracefully (e.g., fallback to cached context data).
    - App should remain operational offline with local storage (SQLite/Room) for tasks and logs.
5. **Security & Privacy**
    - All sensitive data stored locally, anonymized, and encrypted where applicable.
    - Explicit user consent for all context data collection.
    - No personal identifiers shared externally.
6. **Maintainability**
    - Modular code structure for ease of debugging, testing, and future enhancements.
    - Well-documented codebase and clear API boundaries between modules.
7. **Battery & Resource Efficiency**
    - Optimize sensor usage and background processing to minimize battery consumption.
    - Efficient memory and storage utilization for local databases and logs.

7.3 User requirements

User requirements describe **what the end-user expects** from the system.

1. Users must be able to create, view, and manage tasks efficiently with minimal cognitive load.
2. Notifications should be contextually relevant and delivered at moments of low interruption.
3. Users should be able to view task completion history and optional analytics on engagement patterns.
4. The application must operate seamlessly on Android smartphones without requiring constant network connectivity.
5. Users should have control over notification settings, including enabling/disabling alerts or specifying quiet hours.
6. The system should adapt over time to individual user behavior without requiring manual configuration.

7.4 System requirements

System requirements define the **technical specifications and constraints** for the application.

1. **Hardware**
    - Android smartphone (minimum Android 10) with GPS, accelerometer, gyroscope, and sufficient storage (~100MB free).
    - Development computer with Windows, macOS, or Linux to run Android Studio and emulators.
2. **Software**
    - Android Studio IDE for development.
    - Java SDK (Android native app development).
    - SQLite / Room for local data storage.
    - Android APIs for notifications, sensors, and calendar integration.
    - Optional ML libraries (Weka or lightweight Java libraries) for behavior modeling.
3. **Network**
    - Optional internet access for app updates or optional cloud features (analytics sync) but not required for core offline functionality.
4. **Development Tools**
    - Version control: Git + GitHub.
    - Testing: JUnit for unit testing, Espresso for UI testing, Mockito for mocking dependencies.

7.5 Use cases (Tentative)

The following section outlines the primary use cases for the SmartTask system, covering all components and functionalities. These use cases reflect realistic scenarios for end users interacting with the mobile task management application.

7.5.1 Use Case 1: Install and Set Up the Mobile App

- **Actor:** User
- **Description:** The user downloads, installs, and sets up SmartTask on their Android device, including granting necessary permissions and completing initial configuration.
- **Preconditions:** User has an Android smartphone compatible with the app and access to the Google Play Store.
- **Postconditions:** SmartTask is installed, configured, and ready for first-time use.

Main Flow:

1. User searches for “SmartTask” in the Google Play Store.
2. User downloads and installs the app.
3. User opens the app for the first time.
4. App guides the user through initial setup, including permissions for location, motion/activity sensors, calendar access, and notifications.
5. User completes setup, configures notification preferences, and the home dashboard is displayed.

Alternate Flow:

- **Installation Issues:** If storage is insufficient or installation fails, the app shows an error message and instructions to resolve the issue.
- **Permission Denied:** If the user denies required permissions, the app explains limited functionality and prompts to enable them.

7.5.2 Use Case 2: Create and Manage Tasks

- **Actor:** User
- **Description:** The user creates new tasks, adds metadata (deadline, priority, location), and edits or deletes tasks as needed.
- **Preconditions:** SmartTask is installed and running. User has completed initial setup.
- **Postconditions:** Task data is stored in the local database, ready for context-aware prioritization and notification.

Main Flow:

1. User opens the task creation screen.
2. User enters task title, description, deadline, location (optional), and priority.
3. User saves the task.
4. Task metadata is stored in the Task Database.
5. User can view, edit, or delete tasks via the dashboard.

Alternate Flow:

- **Invalid Input:** If mandatory fields are empty or data is invalid (e.g., past date), the app prompts for correction.
- **Task Deletion Failure:** If the database is temporarily inaccessible, deletion is deferred and logged for retry.

7.5.3 Use Case 3: Context Acquisition and Logging

- **Actor:** System
- **Description:** SmartTask continuously collects real-time contextual data, including location, activity, time, calendar events, and device state.
- **Preconditions:** App is running and has required permissions for sensors and calendar access.
- **Postconditions:** Contextual data is stored in the Context Data Store for use in task prioritization and notifications.

Main Flow:

1. System monitors location using Google Maps API geofencing.
2. System collects motion and activity data via motion sensors.
3. System checks calendar events and device state (battery, connectivity).
4. Contextual data is logged in the Context Data Store.

Alternate Flow:

- **Sensor Error:** If a sensor fails, system retries data capture or uses last known values.
- **Permission Revoked:** If user revokes permissions, context logging is paused and the user is notified.

7.5.4 Use Case 4: Dynamic Task Prioritization

- **Actor:** System
- **Description:** SmartTask automatically ranks tasks based on urgency, context relevance, and historical user behavior.
- **Preconditions:** Tasks exist in the database, and contextual data is being captured.
- **Postconditions:** Tasks are dynamically reordered and ready for display and notifications.

Main Flow:

1. System retrieves pending tasks from the Task Database.
2. Context Matching Engine evaluates tasks against current context.
3. Task Prioritization Module calculates urgency and relevance scores using historical behavior logs.
4. Prioritized task list is updated in the dashboard and used for notifications.

Alternate Flow:

- **Missing Context Data:** If context is incomplete, tasks are ordered based on default priority or historical trends.

- **Algorithm Error:** Fallback to last known valid ranking.

7.5.5 Use Case 5: Adaptive Notification Delivery

- **Actor:** System / User
- **Description:** SmartTask delivers reminders or nudges at optimal moments based on context and user behavior to minimize cognitive overload.
- **Preconditions:** Task list is prioritized, notifications are enabled, and context acquisition is active.
- **Postconditions:** Notifications are delivered at appropriate times, and user responses are logged.

Main Flow:

1. System checks task list and context for notification triggers.
2. Notification Engine schedules alerts when user is likely to be receptive (e.g., low activity, free time).
3. User receives and interacts with the notification (mark as done, snooze, or dismiss).
4. User responses are logged for behavioral learning.

Alternate Flow:

- **Notification Suppressed:** If system detects high cognitive load or driving mode, notifications are deferred.
- **User Ignores Notification:** Interaction is logged for future prioritization refinement.

7.5.6 Use Case 6: Behavioral Learning and Insights

- **Actor:** System
- **Description:** SmartTask continuously analyzes user interactions to refine task prioritization and notification timing.
- **Preconditions:** User has interacted with tasks and notifications; sufficient data is logged.

- **Postconditions:** Task and notification algorithms are updated for personalized recommendations.

Main Flow:

1. Data Logging Module aggregates task completions, dismissals, and ignored notifications.
2. Insights Module identifies patterns in user behavior (e.g., preferred times for tasks).
3. Task Prioritization and Notification Modules are updated based on these insights.

Alternate Flow:

- **Insufficient Data:** If user interactions are minimal, the system defaults to rule-based prioritization.
- **Analysis Error:** Errors in analysis are logged, and previous configurations are retained until resolved.

7.6 Test Cases (Tentative)

Table 7.1 displays the test cases used to evaluate the system.

Table 7.1 : Test cases used to evaluate the system and their details

Test Case ID	Use Case	Test Description	Test Steps	Expected Result	Pass/Fail Criteria
TC01	Install & Set Up App	Verify app installation and initial setup	1. Search for SmartTask in Play Store 2. Install the app 3. Open app 4. Complete permissions setup	App installs successfully, initial setup completes, dashboard is displayed	App opens and setup completes without errors
TC02	Install & Set Up App	Handle insufficient storage during installation	1. Attempt installation with <100MB free storage	Installation fails gracefully with error message and instructions	Error message displayed; app does not crash
TC03	Create & Manage Tasks	Verify task creation with valid metadata	1. Open task creation screen 2. Enter title, description, deadline, priority, location 3. Save task	Task saved and visible in dashboard	Task appears in list with correct details

TC04	Create & Manage Tasks	Validate input error handling	1. Leave mandatory fields empty 2. Attempt to save task	App prompts user to fill required fields	User cannot save incomplete tasks
TC05	Create & Manage Tasks	Edit and delete tasks	1. Select task 2. Edit details and save 3. Delete task	Task updates correctly; deletion removes task from dashboard	Task reflects changes; deleted task disappears
TC06	Context Acquisition & Logging	Verify real-time context capture	1. Enable app permissions 2. Move through different locations and activities 3. Trigger task	System logs GPS, activity, time, calendar, and device state correctly	Context data stored accurately in database
TC07	Context Acquisition & Logging	Handle sensor or permission errors	1. Disable location or motion sensors 2. Trigger data logging	System uses fallback or pauses logging, informs user	Logging continues with fallback or displays warning
TC08	Dynamic Task Prioritization	Verify correct task ranking	1. Create multiple tasks 2. Simulate context changes (location, calendar events, activity) 3. Refresh dashboard	Tasks reordered dynamically according to urgency, relevance, and context	Tasks list order matches expected priority

TC09	Adaptive Notification Delivery	Verify notifications trigger at contextually appropriate times	1. Set tasks with reminders 2. Simulate low activity or free time context	Notification delivered at optimal moment	User receives notification when receptive
TC10	Adaptive Notification Delivery	Suppress notifications in inappropriate contexts	1. Simulate user busy or driving 2. Trigger task notification	Notification suppressed or deferred	Notification not shown during high cognitive load
TC11	Behavioral Learning & Insights	Verify learning from user interactions	1. Interact with notifications (complete, dismiss, snooze) 2. Observe prioritization and notification adjustments	System updates task ranking and notification timing	Future prioritization reflects user patterns
TC12	Behavioral Learning & Insights	Handle insufficient user interaction	1. Provide minimal task interaction over time	System maintains default rule-based prioritization	Prioritization continues without errors

TC13	Overall System	Offline operation	1. Disable network 2. Perform task creation, context logging, and notifications	Core features work offline	App operates normally; data synced when online
TC14	Performance	Context processing latency	1. Trigger multiple tasks and context events	System processes context and prioritizes tasks in <1 second	Tasks reordered promptly; notifications delivered timely
TC15	Security & Privacy	Data anonymization and encryption	1. Inspect local database for sensitive identifiers	Sensitive data anonymized/encrypted	No personal identifiers stored unencrypted

8 BUDGET AND BUDGET JUSTIFICATION

Table 8.1 attached below outlines the budget of the project.

Table 8.1 : Budget details of the project

Category	Item / Description	Estimated Cost (USD)	Justification
Labour	Development (UI/UX, backend, ML, testing) – by student	**0**	Student is the developer; no external labour cost.
Material	Android testing devices (student already owns devices)	**0**	Existing devices cover testing; no extra purchase required.
APIs & Services	Google Maps Platform & related APIs usage	**50**	Low-volume usage estimated (~$50) for map/geofence and sensor API calls.
Software & Tools	Figma free tier, Android Studio free, GitHub free	**0**	All required tools are either free or covered under student license.
Deployment & Hosting	Google Play Developer account fee	**25**	One-time fee to publish the app on Google Play.
Transportation & Miscellaneous	Travel / transit for field testing + incidental costs (data usage, test site coffee, etc.)	**30**	Estimate: ~3 field testing excursions × ≈$10 each for transit/data.
Total Estimated Cost		**$105**	

9 ABILITY OF COMMERCIALIZATION

- **Differentiation:** SmartTask stands out from conventional task managers because it adapts to real-life context, taking into account location, user activity, and calendar events, while intelligently prioritizing tasks. This provides highly personalized productivity support that fixed-time reminder apps cannot offer.
- **Investment & Cost:** Development costs are minimal as the app is created by a student with no labor cost, using existing Android devices with no hardware cost. The main expense is API subscriptions, such as Google Maps costing approximately USD 50 per year for geofencing. Revenue is expected through a freemium model with free ad-supported access and premium subscriptions offering advanced insights and an ad-free experience.
- **Target Market & Customer Profile:** SmartTask targets students, busy professionals, and productivity enthusiasts. These users are mobile-first, heavily rely on task lists, and operate in fast-paced environments. The app helps them improve efficiency without adding cognitive load.
- **Value Proposition:** SmartTask reduces cognitive strain by delivering timely, context-aware notifications rather than generic reminders. This helps users complete tasks on time, manage priorities dynamically, and increase overall productivity in their daily routines.
- **Intellectual Property Considerations:** The proprietary context-matching and adaptive notification algorithms create defensible intellectual property. This prevents competitors from easily replicating SmartTask's core functionality and strengthens its long-term commercial potential.

10 REFERENCES

[1] G. Mark, D. Gudith, and U. Klocke, “The cost of interrupted work: More speed and stress,” *in Proc. ACM CHI*, 2008, pp. 107–110.

[2] V. Gonzalez, V. M. Gonzalez, and G. Mark, “Constant, constant, multitasking craziness: Managing multiple working spheres,” *in Proc. ACM CHI*, 2004, pp. 113–120.

[3] H. Winghart and A. Al Zaidy, “MageMemo: A gamified productivity tool for user engagement,” *J. Inf. Technol., Cybersecurity, and AI*, vol. 1, no. 1, pp. 9–17, 2024.

[4] A. K. Dey, “Understanding and using context,” *Personal and Ubiquitous Computing*, vol. 5, no. 1, pp. 4–7, 2001.

[5] M. Baldauf, S. Dustdar, and F. Rosenberg, “A survey on context-aware systems,” *Int. J. Ad Hoc and Ubiquitous Computing*, vol. 2, no. 4, pp. 263–277, 2007.

[6] A. González-Pérez, I. Herrera-Carnicer, and R. Contreras, “AwarNS: A framework for developing context-aware reactive mobile applications for health and mental health,” *J. Biomed. Inform.*, vol. 141, 2023, Art. 104359.

[7] T. Sohn, A. Varshavsky, D. Chen, R. K. Gupta, and W. G. Griswold, “Place-its: A study of location-based reminders on mobile phones,” *in Proc. ACM UbiComp*, 2005, LNCS 3660, pp. 232–250.

[8] A. Kessell and C. Chan, “Castaway: A context-aware task management system,” *in CHI 2006 Ext. Abstracts*, 2006, pp. 941–946.

[9] B. M. Landry *et al.*, “Taskminder: A context- and user-aware to-do list management system,” Tech. Rep., Georgia Institute of Technology, 2003.

[10] P. Muñoz, P. Giner, and M. Gil, “Designing context-aware interactions for task-based applications,” *in Proc. Int. Conf. on Web Engineering (ICWE)*, 2010, LNCS 6385, pp. 463–473.

[11] B. Chihani, E. Bertin, and N. Crespi, “A user-centric context-aware mobile assistant,” *in Proc. IEEE Int. Conf. on Intelligence in Next Generation Networks (ICIN)*, 2013, pp. 17–21.

[12] C. Zhang *et al.*, “Grounded task prioritization with context-aware sequential ranking,” *ACM Trans. Inf. Syst.*, vol. 40, no. 4, Art. 28, 2022.

[13] Todoist, “A to-do list to organize your work & life,” 2025. [Online]. Available: https://www.todoist.com

[14] Todoist, “Use location reminders in Todoist,” updated Sep. 23, 2025. [Online]. Available: https://www.todoist.com/help/articles/use-location-reminders-in-todoist-uGcwH2AJ6

[15] Any.do, “How to manage notifications in Any.do,” 2023. [Online]. Available: https://support.any.do/en/articles/8610107-how-to-manage-notifications-in-any-do

[16] Microsoft, “Microsoft To Do: Task Management App,” 2025. [Online]. Available: https://www.microsoft.com/en-us/microsoft-365/microsoft-to-do

[17] Google, “Manage Google Tasks notification settings,” 2025. [Online]. Available: https://support.google.com/tasks/answer/13486720

[18] A. Li, “How Google Keep’s switch to Tasks will work: No location reminders,” *9to5Google*, Sep. 6, 2025. [Online]. Available: https://9to5google.com/2025/09/06/google-keep-tasks-switch-locaton

[19] TickTick, “An introduction to TickTick,” Aug. 2, 2016. [Online]. Available: https://blog.ticktick.com/2016/08/02/an-introduction-to-ticktick

[20] Trello, “Capture, organize, and tackle your to-dos from anywhere,” 2025. [Online]. Available: https://trello.com

[21] F. Luqman and M. L. Griss, “Overseer: A mobile context-aware collaboration and task management system for disaster response,” *in Proc. IEEE C5*, 2010.

[22] M. Streefkerk, G.-J. van der Linden, and G. de Haan, “Do(not)Disturb: Effects of context-aware notification management on task performance,” *Int. J. Hum.-Comput. Stud.*, vol. 70, no. 3, pp. 239–245, 2012.

[23] S. T. Iqbal, *A Framework for Intelligent Notification Management in Multitasking Domains*, Ph.D. dissertation, Univ. Illinois at Urbana-Champaign, 2008.

[24] Q. U. Ahmed and S. B. Mujib, “Context aware reminder system: Activity recognition using smartphone accelerometer and gyroscope,” M.Sc. thesis, Blekinge Inst. Technol., Sweden, 2014.

[25] O. A. Faruque, M. Koehl, F. L. Goh *et al.*, “AWARE: A mobile context instrumentation framework,” *Front. ICT*, vol. 2, Art. 6, 2015.

[26] Y. Tian, Z. Wang, D. Pelleg *et al.*, “Characterization and prediction of mobile tasks,” *ACM Trans. Inf. Syst.*, vol. 41, no. 1, 2022.

[27] I. Kim, H. Goh, N. Narziev, Y. Noh, and U. Lee, “Understanding user contexts and coping strategies for context-aware phone distraction management system design,” *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 4, no. 4, Art. 134, 2020.

[28] B. P. Bailey and J. A. Konstan, “Are informational notifications disruptive?,” *in Proc. ACM CHI*, 2006, pp. 331–340.